

Depth, Oversmoothing, and Architecture: A Comparative Study of Graph Neural Networks on the Cora Citation Network

Project Proposal (Report 1)
CSE 705 Deep Learning - Summer 2026

Kiarash Ara and Yiheng Wu

1. Problem Specification and Motivation

Graph-structured data appears across many domains, and learning vector representations of the nodes in a graph is a core problem in modern deep learning. We study *semi-supervised node classification* on the Cora citation network, a standard benchmark in which 2,708 machine-learning papers form the nodes and undirected citation links form the edges. Each node carries a 1,433-dimensional binary bag-of-words feature vector and belongs to one of seven topic classes. Only a small fraction of nodes is labeled during training (the standard split uses 140 labeled nodes), so the task is to propagate label information across the graph structure to classify the remaining nodes.

The problem is primarily *applied and empirical*, but it is organized around a theoretical question. A vanilla graph convolutional network already solves basic Cora classification at shallow depth, so simply reproducing that result would be uninteresting. We instead investigate a known failure mode of message-passing GNNs: *oversmoothing*, the tendency of repeated neighborhood aggregation to drive all node representations toward an indistinguishable fixed point as network depth grows. Depth is not a nuisance parameter. It controls how far across the graph a node can draw information from. Understanding why deeper GNNs frequently perform *worse*, and which techniques restore deep performance, is therefore a central question in graph representation learning.

Our motivation is twofold. First, node classification underpins the real-world applications the project asks us to consider: recommendation systems (user-item graphs), social-network analysis (community and role detection), and drug discovery (molecular property prediction), all of which depend on learning discriminative node or subgraph representations. Second, one of us conducts research on physics-informed graph neural networks, where depth is not optional: propagating physical state across a discretized domain requires information to reach across enough of that domain, which makes oversmoothing a first-order obstacle. Cora is a small, tightly clustered citation graph and will *not* transfer domain-specific findings to physics meshes, which are far larger and require information to travel much farther. What transfers is the *diagnostic methodology and*

the qualitative behavior of the mitigations, a reusable measurement apparatus, not the specific numbers.

2. Methodology

All three architectures we compare belong to the same family of *message-passing* graph neural networks: each layer updates a paper’s representation by gathering information from its neighbors in the citation graph and combining it with the paper’s own current representation. Stacking such layers lets information travel farther across the graph. One layer reaches direct neighbors, two layers reach neighbors-of-neighbors, and so on. The three architectures differ in how they gather and weight that neighbor information:

- **GCN (Kipf & Welling, 2017):** averages each node’s neighbors in a fixed, structure-determined way, treating all neighbors symmetrically.
- **GraphSAGE (Hamilton et al., 2017):** samples a subset of neighbors and keeps a node’s own features separate from its neighbors’ before combining them, which makes it scalable and able to generalize to nodes unseen during training.
- **GAT (Veličković et al., 2018):** uses an attention mechanism to learn how much weight to give each neighbor, so that more relevant neighbors contribute more.

Across all three we increase network *depth* (from 2 up to 32 layers) and track two things: how accurately the model classifies papers, and how *distinct* the learned representations remain. As depth grows, message-passing networks tend to blur node representations together until they become nearly identical, the *oversmoothing* problem, at which point accuracy collapses. We measure this collapse with two standard indicators: *Dirichlet energy* (how much neighboring nodes’ representations still differ, which falls toward zero as they blur together) and *mean average distance* (MAD, the average dissimilarity across all node representations). We record these at every layer, not only at the output, so we can see where in the network the blurring begins. We then test whether established fixes restore performance at depth: residual/skip connections, PairNorm (Zhao & Akoglu, 2020), Jumping-Knowledge aggregation (Xu et al., 2018), and GCNII (Chen et al., 2020). Training uses standard choices: the Adam optimizer, dropout, weight decay, and early stopping on validation accuracy.

3. Testing and Experimentation

Dataset and protocol. We use the standard benchmark split of Cora (Yang et al., 2016): 140 papers for training, 500 for validation, and 1,000 for testing. To control for random variation, every configuration is run over 10 random seeds and we report the mean and standard deviation.

Metrics. We report classification accuracy, micro-F1, and macro-F1. We note explicitly that for single-label multiclass classification, micro-F1 is mathematically equal to accuracy; we therefore

add *macro*-F1 to capture per-class performance under Cora’s class imbalance, rather than reporting a redundant metric. Oversmoothing is tracked separately via Dirichlet energy and MAD.

Planned experiments:

1. **Baseline reproduction:** each architecture at 2 layers, confirming we match published Cora accuracy (81-83%).
2. **Depth sweep:** accuracy and both smoothing diagnostics as a function of depth, per architecture, to locate the onset of representation collapse.
3. **Mitigation ablations:** apply the full mitigation set (residual, PairNorm, JK, GCNII) to GCN across the depth range, then carry the single most effective mitigation to GraphSAGE and GAT to test whether the fix generalizes across architectures.
4. **Qualitative analysis:** two-dimensional visualizations (t-SNE / UMAP) of the learned node representations at shallow vs. deep settings, showing the transition from well-separated topic clusters to a single collapsed cluster in which the seven classes are no longer separable.
5. **Hyperparameter study:** learning rate, hidden dimension, dropout, and weight decay tuned on the validation set only.

Scope. The experiment grid is deliberately sized for the four-week timeline: the depth sweep is capped at 32 layers (collapse is fully evident well before this depth on a graph as small as Cora), and the full mitigation set is evaluated only on GCN, with the strongest mitigation carried over to GraphSAGE and GAT. This keeps the number of distinct configurations tractable while still supporting every claim we intend to make.

Large-scale and noisy graphs. We also discuss the challenges of scale and noise: we relate GraphSAGE’s neighbor sampling to training on graphs too large for full-batch computation, and connect our oversmoothing findings to a concrete improvement proposal, depth-aware architecture design with normalization, as a defense against both over-aggregation and structural (edge) noise.

4. Project Plan and Milestones

Roles. Kiarash (integration owner) is responsible for the repository and experiment harness, the three architecture implementations, the depth-sweep infrastructure, and the training/evaluation loop. Yiheng Wu is responsible for the oversmoothing metrics (Dirichlet energy, MAD), embedding visualization, the hyperparameter search, and the applications / large-scale-and-noise discussion. Both members co-lead the analysis and jointly write the report and presentation.

Phase	Tasks	Lead
Jul 7-13	Data pipeline; reproduce 2-layer GCN, GraphSAGE, GAT baselines.	Kiarash

Phase	Tasks	Lead
Jul 14-20	Depth-sweep harness; implement + validate Dirichlet energy and MAD; run full depth sweep.	Kiarash (harness) / Yiheng (metrics)
Jul 21-27	Mitigation ablations (residual, PairNorm, JK, GCNII); hyperparameter study; embedding visualization.	Kiarash (ablations) / Yiheng (tuning + viz)
Jul 28-Aug 3	Qualitative analysis; applications & noise discussion; final report and presentation (due Aug 3).	Both

5. References

1. Kipf, T. N., & Welling, M. (2017). Semi-Supervised Classification with Graph Convolutional Networks. ICLR.
2. Hamilton, W. L., Ying, R., & Leskovec, J. (2017). Inductive Representation Learning on Large Graphs (GraphSAGE). NeurIPS.
3. Veličković, P., Cucurull, G., Casanova, A., Romero, A., Liò, P., & Bengio, Y. (2018). Graph Attention Networks. ICLR.
4. Li, Q., Han, Z., & Wu, X.-M. (2018). Deeper Insights into Graph Convolutional Networks for Semi-Supervised Learning. AAAI.
5. Xu, K., Li, C., Tian, Y., Sonobe, T., Kawarabayashi, K., & Jegelka, S. (2018). Representation Learning on Graphs with Jumping Knowledge Networks. ICML.
6. Zhao, L., & Akoglu, L. (2020). PairNorm: Tackling Oversmoothing in GNNs. ICLR.
7. Chen, M., Wei, Z., Huang, Z., Ding, B., & Li, Y. (2020). Simple and Deep Graph Convolutional Networks (GCNII). ICML.
8. Yang, Z., Cohen, W. W., & Salakhutdinov, R. (2016). Revisiting Semi-Supervised Learning with Graph Embeddings (Planetoid). ICML.
9. Sen, P., Namata, G., Bilgic, M., Getoor, L., Gallagher, B., & Eliassi-Rad, T. (2008). Collective Classification in Network Data. AI Magazine, 29(3).
10. Fey, M., & Lenssen, J. E. (2019). Fast Graph Representation Learning with PyTorch Geometric. ICLR Workshop on Representation Learning on Graphs and Manifolds.